

Monte Carlo Simulation of Polyethylene Chains

Implementation Notes

ManelDC55

Contents

1	Serial Monte Carlo Engine	2
1.1	The Pivot Move and Rodrigues' Rotation	2
1.2	Convergence Detection: the Geweke z -Test	2
2	Parallel Equilibration via MPI	4
2.1	Strategy: Collaborative Population Equilibration	4
2.2	Replica Selection: Boltzmann Roulette Wheel	4
2.3	MPI Communication Protocol	4
3	Parallelisation Efficiency and Sampling Results	6
3.1	Metrics and Methodology	6
3.2	Convergence Efficiency and Superlinear Speedup	6
3.3	Discussion	6

1 Serial Monte Carlo Engine

1.1 The Pivot Move and Rodrigues' Rotation

The core conformational sampling mechanism is the pivot move, implemented within the `rotate_dihedral` subroutine. Rather than performing local perturbations, the pivot move selects a random bond along the chain and rotates the entire subsequent segment (the “tail”) as a rigid body..

To perform the rotation without distorting bond lengths or bond angles, Rodrigues' rotation formula is used. Given a vector $\vec{v}$ representing an atom's position relative to the pivot point, a unit rotation axis $\hat{k}$ (the chosen C–C bond direction), and a rotation angle ϕ , the rotated vector is:

$$\vec{v}_{\text{rot}} = \vec{v} \cos \phi + (\hat{k} \times \vec{v}) \sin \phi + \hat{k} (\hat{k} \cdot \vec{v}) (1 - \cos \phi) \quad (1)$$

The formula decomposes the rotation into three contributions: a scaled version of the original vector, a cross-product term that generates the perpendicular component, and a projection term that preserves the component parallel to the axis.

When explicit hydrogens are included (all-atom mode), the rotation must also displace the two hydrogens attached to each rotating carbon. Because of the sp^3 hybridisation, both hydrogens must rotate with exactly the same axis and angle as their parent carbon; any deviation would alter C–H bond lengths or C–C–H angles, producing non-physical geometries that would be immediately rejected by the Metropolis criterion.

The implementation identifies each hydrogen by checking whether the distance to a rotating carbon falls below the C–H bond threshold of 1.2 Å, then applies Eq. (1) with the same pivot, axis, and angle:

```
if (explicit_h) then
  ! Hydrogens are stored after the carbon backbone in the coords array
  do i = n_carbons + 1, n_atoms
    do j = k + 1, n_carbons
      ! Identify if hydrogen i is bonded to the moving carbon j
      v = coords(i, :) - coords(j, :)
      if (vnorm(v) < 1.2d0) then
        ! Apply Rodrigues' rotation relative to the pivot point
        v = coords(i, :) - pivot
        dot_uv = sum(axis * v)
        v_rot = v * cos_p + cross(axis, v) * sin_p &
          + axis * dot_uv * (1.0d0 - cos_p)
        coords_new(i, :) = pivot + v_rot
        exit ! Hydrogen found, move to the next atom
      end if
    end do
  end do
end if
```

Listing 1: Pivot move — hydrogen rotation in AA mode.

1.2 Convergence Detection: the Geweke z -Test

Detecting equilibration of a Markov chain is non-trivial because successive samples are highly correlated. The Geweke diagnostic was used, as it is designed specifically for Markov chain output and accounts for this autocorrelation.

The test compares the mean energy in the first $f_A = 10\%$ of an accumulated buffer of $N = 300$ energy samples against the mean in the last $f_B = 50\%$:

$$z = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{SE_A^2 + SE_B^2}} \quad (2)$$

Here, SE_A and SE_B represent the Standard Errors of the mean energy for window A and window B, respectively. They measure the statistical uncertainty of our calculated averages.

In a dataset of independent measurements, calculating the standard error is straightforward. However, Monte Carlo trajectories are autocorrelated—each new geometry is simply the previous one with a slight rotation. If we computed SE_A and SE_B directly from the raw sample variance, the autocorrelation would cause us to drastically underestimate the error, tricking the algorithm into declaring a false equilibrium.

To solve this, the code computes SE_A and SE_B using batch means. Instead of treating all n samples in a window individually, the window is divided into $\lfloor \sqrt{n} \rfloor$ discrete batches or blocks. By calculating the variance across the averages of these batches, the local autocorrelation is "smoothed out," providing a consistent and conservative estimator of the true long-run variance.

Under the null hypothesis of stationarity, z follows a standard normal distribution. Equilibrium is accepted when $|z| < z_{\text{crit}} = 1.96$ (95% confidence level) on $n_{\text{consec}} = 3$ consecutive evaluations, preventing false positives caused by temporary plateaus. The test is re-evaluated every `eval_freq` synchronisation points once the buffer is full.

```
! Batch means SE for window A (first fA% of buffer)
nA      = max(2, n_geweke * fA_pct / 100)    ! = 30 samples
bA      = max(2, int(sqrt(dble(nA))))        ! number of batches
bsA     = nA / bA                            ! batch size
seA_E   = 0.0d0
do ib = 1, bA
  bm     = sum(tmp_E((ib-1)*bsA+1 : ib*bsA)) / dble(bsA)
  seA_E  = seA_E + (bm - meanA_E)**2
end do
seA_E   = seA_E / dble(bA * (bA - 1))        ! SE^2 of window A mean

! Geweke z-statistic
z_E     = abs(meanA_E - meanB_E) / sqrt(seA_E + seB_E)

if (z_E < z_crit) then    ! z_crit = 1.96 (95% confidence)
  consec_passes(task) = consec_passes(task) + 1
  if (consec_passes(task) >= n_consec) equil_done(task) = .true.
else
  consec_passes(task) = 0 ! reset counter on failure
end if
```

Listing 2: Batch-means SE and Geweke z -statistic.

The radius of gyration R_g is monitored with its own Geweke z -statistic and reported in the output, but is deliberately excluded from the stopping criterion. In practice, R_g was found to equilibrate at a different rate than the energy and would prevent convergence from being declared even when the energy had clearly stabilised.

2 Parallel Equilibration via MPI

2.1 Strategy: Collaborative Population Equilibration

Rather than assigning a single worker to equilibrate each configuration type independently, a group of `workers_per_equil` MPI workers is assigned to each configuration type simultaneously. Each worker starts from the same initial geometry but with a different random seed (`rng_seed + rank × 104729`), ensuring that trajectories immediately diverge and explore distinct regions of conformational space.

Every `sync_interval` MC steps, a synchronisation protocol is executed: each worker sends its current energy and R_g to the master, which selects the most representative configuration via a Boltzmann-weighted roulette wheel and broadcasts those coordinates back to the non-winning workers in the group. The winning worker continues its trajectory uninterrupted, preserving the best conformational path found so far.

2.2 Replica Selection: Boltzmann Roulette Wheel

At each synchronisation point, the master assigns a statistical weight to each replica i based on its total energy E_i :

$$w_i = \exp\left(-\frac{E_i - E_{\min}}{k_B T_{\text{virt}}}\right) \quad (3)$$

where $E_{\min}$ is the lowest energy in the current group and T_{virt} is a virtual temperature — a purely algorithmic parameter with no physical meaning. It controls how aggressively low-energy replicas are favoured:

- $T_{\text{virt}} \rightarrow 0$: deterministic selection of the minimum-energy replica.
- $T_{\text{virt}} \rightarrow \infty$: uniform selection, ignoring energies.

A uniform random number then samples this distribution via roulette-wheel selection, and the selected replica's coordinates are sent to the non-winning workers:

```
E_min_grp = minval(grp_energies)
do w = 1, workers_per_equil
  grp_boltz(w) = exp(-(grp_energies(w) - E_min_grp) / (kb * T_virt))
end do
tw = sum(grp_boltz)
call random_number(r_pick)
accum = 0.0d0
best_local = workers_per_equil
do w = 1, workers_per_equil
  accum = accum + grp_boltz(w) / tw
  if (r_pick <= accum) then
    best_local = w
    exit
  end if
end do
```

Listing 3: Boltzmann roulette wheel selection.

2.3 MPI Communication Protocol

The synchronisation uses only point-to-point MPI operations (`MPI_Send` / `MPI_Recv`) to avoid the need for temporary communicators or collective operations across worker subgroups. At each

sync point, the master loops over all K workers in the active equilibration group following the protocol described in Table 1.

Table 1: Point-to-point MPI synchronisation protocol at each `sync_interval`.

Step	Direction	Tag	Content
1	worker $\rightarrow$ master	TAG_SYNC_OBS	$[E_{\text{total}}, R_g^2]$
2	master $\rightarrow$ worker	TAG_SYNC_CTRL	<code>[ctrl_signal, best_local_idx]</code>
3	worker $\rightarrow$ master	TAG_EQUIL_COORDS	full coordinate array
4	master $\rightarrow$ non-winners	TAG_SYNC_COORDS	winning coordinates

Once the Geweke test (Section 1.2) declares equilibrium, the master saves the equilibrated geometry to `../results/equilibrated_cX.xyz` and unlocks the 10 production jobs for that configuration type in the task queue.

3 Parallelisation Efficiency and Sampling Results

3.1 Metrics and Methodology

This section evaluates the performance of the collaborative parallel implementation compared to the serial baseline. The system studied the configuration 1 chain of 500 carbons at $T = 300$ K including hydrogens

We evaluate efficiency based on the time to reach specific energy thresholds ($E_{total} < 110, 100$, and 90 kcal/mol). The speedup is defined as $S(w) = \text{Steps}_{\text{serial}} / \text{Steps}_{\text{parallel}}(w)$.

3.2 Convergence Efficiency and Superlinear Speedup

Table 2 summarises the number of MC steps required by different worker configurations to reach the target energy levels.

Table 2: Sampling efficiency: MC steps and speedup to reach energy thresholds.

Threshold	Workers (w)	Steps Required	Speedup $S(w)$	Efficiency $S(w)/w$
$E < 110$ kcal/mol	1	590,000	1.00	1.00
	2	70,000	8.43	4.21
	4	140,000	4.21	1.05
	8	50,000	11.80	1.47
$E < 100$ kcal/mol	1	1,180,000	1.00	1.00
	2	350,000	3.37	1.68
	4	210,000	5.62	1.40
	8	50,000	23.60	2.95
$E < 90$ kcal/mol	1	2,400,000	1.00	1.00
	2	1,780,000	1.35	0.67
	4	570,000	4.21	1.05
	8	160,000	15.00	1.87

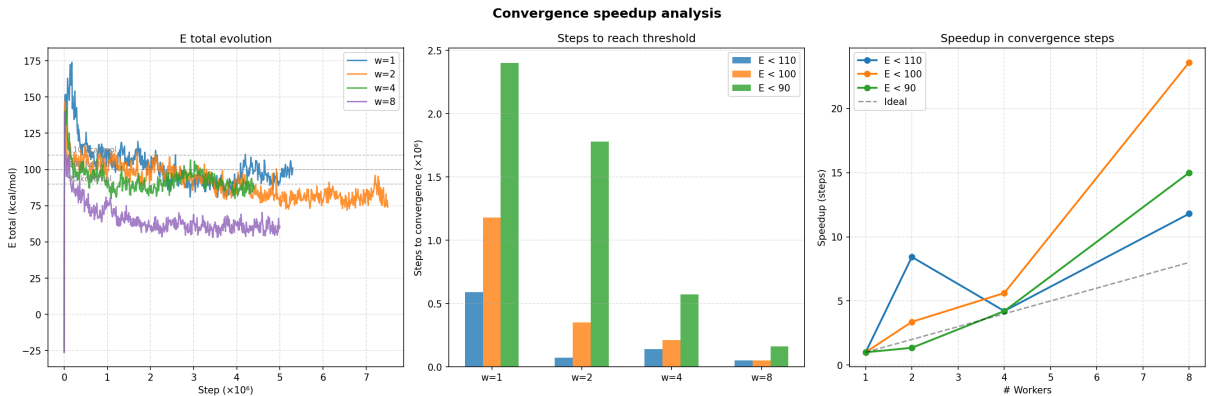


Figure 1: Convergence speedup analysis. *Left*: evolution of E_{total} for each worker configuration. *Centre*: MC steps required to reach each energy threshold. *Right*: speedup $S(w)$ relative to the serial run; the dashed line shows ideal linear scaling.

3.3 Discussion

The results demonstrate a clear superlinear speedup ($S(w) > w$) in almost all parallel configurations. This phenomenon is a direct consequence of the collaborative sampling strategy

implemented via the Boltzmann roulette wheel.

In the serial run, the polymer chain often becomes trapped in metastable states (local energy minima). The parallel implementation, by maintaining a population of w independent trajectories, significantly increases the probability that at least one replica discovers a more favorable region of the phase space. Once a lower-energy conformation is found, the synchronization protocol "rescues" the rest of the population, broadcasting the winning coordinates to all workers. This creates a collective escape mechanism that reduces the number of steps required to converge by a factor far greater than the number of processors.

For instance, at the 100 kcal/mol threshold, the 8-worker group reaches the target in only 50,000 steps, compared to the 1.18 million steps required by the serial run, resulting in a speedup of **23.60x**.

We also observe stochastic fluctuations typical of Monte Carlo methods, such as the anomalous efficiency at $w = 2$ for the 110 kcal/mol threshold. As the energy targets become more stringent (90 kcal/mol), the speedup values tend to stabilize, as finding further conformational improvements becomes more difficult even for a larger population.